Linux Shell Programming Cheatsheet

CDAC CCEE Complete Revision Guide

1 1. Shell Basics & Startup

1.1 The Shell

A command interpreter that acts as an interface between the User and the Kernel.

- **Common Shells:** `bash` (default), `sh`, `csh`, `ksh`.
- **Shebang:** `#!/bin/bash`
  (Must be the very first line of the script).

1.2 Startup Sequence (Important)

1. `/etc/profile`: Global config (Runs first for everyone).
2. `~/.bash_profile`: Login Shells only (SSH, GUI Login).
3. `~/.bashrc`: Non-Login Interactive Shells (Opening a new terminal window).
4. `~/.bash_logout`: Runs when exiting a login shell.

2 2. Variables & Quotes

2.1 Assignment Rules

Rule: No spaces allowed around the = sign.

```
var="Hello"  Valid
var = "Hello"  Invalid (Space error)
```

2.2 Quoting Mechanisms

- **Double Quotes (" "):** Weak quotes. Variables **are expanded**.

```
echo "My name is $name" -> My name is John
```

- **Single Quotes (' '):** Strong quotes. Everything is treated literally.

```
echo 'My name is $name' -> My name is $name
```

- **Backticks (` `):** Command Substitution (Old style). `date=`date``
- **$(...):** Command Substitution (Modern style). `date=$(date)`

3 3. Positional Parameters

Used to handle command line arguments.
Example: `./script.sh apple banana cherry`

$0 Name of the script (`./script.sh`)

$1 First argument (`apple`)

$2 Second argument (banana)

$# Total count of arguments (3)

$@ All arguments as a list (`apple banana cherry`)

$* All arguments as a single string

$? Exit Status of previous command
(**0** = Success, **1-255** = Error)

$$ PID of the current script

4 4. Arithmetic Operations

Bash variables are strings by default.

4.1 Method 1: expr (Old)

- Requires spaces around operators.
- Requires escaping specific symbols like *.

```
expr 5 \* 2  Valid (Output: 10)
expr 5 * 2  Invalid (Syntax Error)
expr 5+2  Invalid (String "5+2")
```

4.2 Method 2: Double Parentheses (Best)

- No spaces required. Standard C-style math.

```
res=$((5 + 2))
(( i++ ))  # Increment variable
```

4.3 Method 3: let

```
let sum=5+2
```

5 5. Logic & Control Flow

5.1 Conditional: if-else

CRITICAL: Spaces are mandatory inside [ ].

```
if [ $x -gt 10 ]; then
   echo "High"
elif [ $x -eq 10 ]; then
   echo "Equal"
else
   echo "Low"
fi
```

5.2 Comparison Operators

Type	Operator	Meaning
Integer	-eq	Equal
	-ne	Not Equal
	-gt / -lt	Greater / Less Than
	-ge / -le	Greater/Eq / Less/Eq
String	=	Equal
	!=	Not Equal
	-z	Empty (Null)
	-n	Not Empty
File	-f	File exists
	-d	Directory exists
	-r	Readable
	-w	Writable
	-x	Executable

5.3 Loops

While Loop (Runs while True):

```
while [ $count -gt 0 ]; do
   echo $count
   ((count--))
done
```

Until Loop (Runs while False):

```
until [ $count -eq 0 ]; do
   # Runs until count becomes 0
   ((count--))
done
```

For Loop (Two Styles):

```
# Style 1: List
for name in Ali Bob Cat; do
   echo $name
done

# Style 2: C-Style
for (( i=0; i<5; i++ )); do
   echo $i
done
```

5.4 Case Statement

```
case $opt in
   start) echo "Starting" ;;
   stop) echo "Stopping" ;;
   *) echo "Invalid" ;;
esac
```

6 6. Filters (Grep, Sed, Awk)

Essential for MCQs involving text processing.

6.1 grep (Search)

Searches for patterns in files.

- `grep "error" log.txt`: Find lines with "error".
- `grep -i "error"`: Case insensitive.
- `grep -v "error"`: Invert match (lines **without** error).
- `grep -c "error"`: Count occurrences.

6.2 sed (Stream Editor)

Used for find and replace.

- **Syntax:** `sed 's/find/replace/flags' file`
- `sed 's/unix/linux/' file`: Replace first occurrence per line.
- `sed 's/unix/linux/g' file`: Replace global (all) occurrences.
- `sed -n '1,5p' file`: Print lines 1 to 5.

6.3 awk (Text Processing)

Works on columns (fields). Default delimiter is space.

- `$1, $2`: First and second column.
- `$0`: The entire line.
- NF: Number of Fields (columns).
- NR: Number of Records (line number).
- **Example:** Print 1st column of file:
  `awk '{print $1}' file.txt`

7 7. File Permissions

7.1 Symbolic Mode (chmod)

`chmod [user][op][perm] file`

- **Users:** u (user), g (group), o (others), a (all).
- **Ops:** + (add), - (remove), = (set).
- **Perms:** `r` (read), `w` (write), x (execute).
- **Example:** `chmod u+x script.sh`

7.2 Octal Mode (chmod)

4 = Read (`r`)
2 = Write (`w`)
1 = Execute (x)
0 = No permission

Example: 755
User: $4+2+1=7$ (rwx)
Group: $4+0+1=5$ (r-x)
Others: $4+0+1=5$ (r-x)

8 8. Process Management

- **`ps`**: Report snapshot of current processes.
- **`ps -ef`**: Show standard syntax (all processes).
- **`top`**: Display real-time system processes.
- **`kill [PID]`**: Terminate a process.
- **`kill -9 [PID]`**: Force kill (SIGKILL).
- **`bg`**: Send job to background.
- **`fg`**: Bring job to foreground.
- **`jobs`**: List active jobs.

9 9. Quick Exam Tips

1. **Spaces matter!** `[ $a -eq $b ]` needs spaces.
2. **Exit Code 0 is Success.** In C, 0 is often "false", but in Bash, 0 means "No Error".
3. **Comparison traps:** Don't use > for numbers inside `[ ]`. Use `-gt`.
4. **Variable Assignment:** `i++` only works in `(( ))` or `let`. `i=i+1` is just a string without `let`.